

Chapter 6e

Balanced Binary Tree(AVL)

平衡二叉树

教材9.2节

平衡二叉树(AVL)

BST是一种查找效率比较高的组织形式，平均查找长度受树的形态影响较大，形态比较均匀时查找效率很好，形态明显偏向某一方向时其效率就大大降低。

更好的二叉排序树，其形态均衡的，查找时能得到最好的效率。

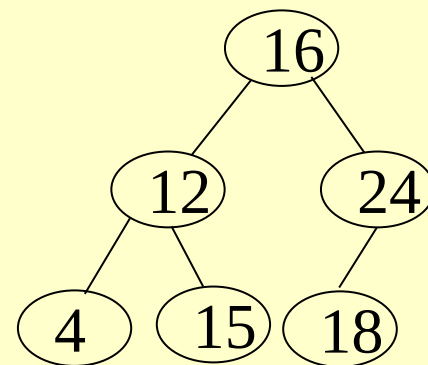
平衡二叉排序树(**Balanced Binary Tree**或**Height-Balanced Tree**)是在1962年由**Adelson-Velskii**和**Landis**提出的，又称**AVL**树。

平衡二叉树的定义

- **平衡二叉树**：是空树，或者满足下列性质的二叉树。
 - (1) 左子树和右子树深度之差的绝对值不大于1；
 - (2) 左子树和右子树也都是平衡二叉树。
- **平衡因子(Balance Factor)**：二叉树上结点的左子树的深度减去其右子树深度称为该结点的平衡因子。
- **平衡二叉树上每个结点的平衡因子只可能是-1、0和1**只要有一个结点的平衡因子的绝对值大于1，该二叉树就不是平衡二叉树。
- **平衡二叉排序树(Balanced Binary Sort Tree)**：一棵二叉树既是二叉排序树又是平衡二叉树。
- 高度为h的平衡二叉树最少结点数： $n_h = 1 + n_{h-1} + n_{h-2}$
 $n_0 = 0, n_1 = 1, n_2 = 2$

结点类型定义：

```
typedef struct BSTNode
{   ElemType data;   /* 关键字域 */
    int bf;          /* 平衡因子域 */
    ...              /* 其它数据域 */
    struct BNode *lchild, *rchild;
} *BSTree, BSTNode;
```



平衡二叉树

- 查找过程与二叉排序树上的查找过程完全相同，则在AVL树上执行查找时，与给定的K值比较的次数不超过树的深度。
- 平衡二叉排序树上进行查找的**平均查找长度**和 $\log_2 n$ 是一个数量级的，平均时间复杂度为 $O(\log_2 n)$ 。

平衡化旋转

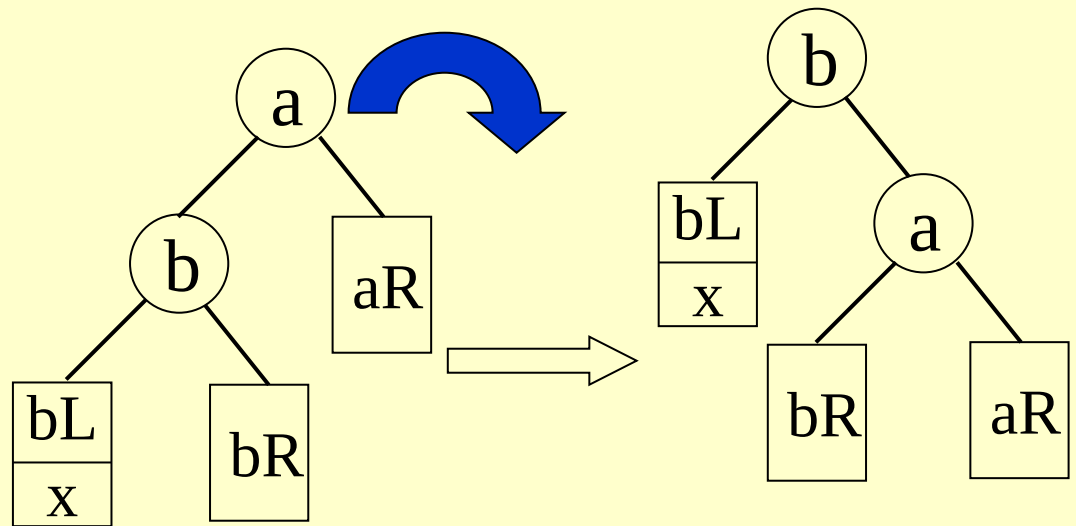
- 一般的二叉排序树是不平衡的，若能通过某种方法使其**既保持有序性，又具有平衡性**，找到了构造平衡二叉排序树的方法，该方法称为**平衡化旋转**。
- 在对AVL树进行插入或删除一个结点后，通常会影响到**从根结点到插入(或删除)结点的路径上的某些结点**，这些结点的子树可能发生变化。以插入结点为例，影响有以下几种可能性：
 - ◆ 以某些结点为根的子树的深度发生了变化；
 - ◆ 某些结点的平衡因子发生了变化；
 - ◆ 某些结点失去平衡。
- **失衡结点**：沿着插入结点上行到根结点就能找到某些结点，这些结点的平衡因子和子树深度都会发生变化。

1 LL型平衡化旋转

(1) 失衡原因

在结点a的左孩子的左子树上进行插入，插入使结点a失去平衡。a插入前的平衡因子是1，插入后的平衡因子是2。设b是a的左孩子，b在插入前的平衡因子只能是0，插入后的平衡因子是1(否则b就是失衡结点)。

(2) 平衡化旋转方法: 顺时针旋转操作实现。右旋用b取代a的位置，a成为b的右子树的根结点，b原来的右子树作为a的左子树。(借)



LL型平衡化旋转

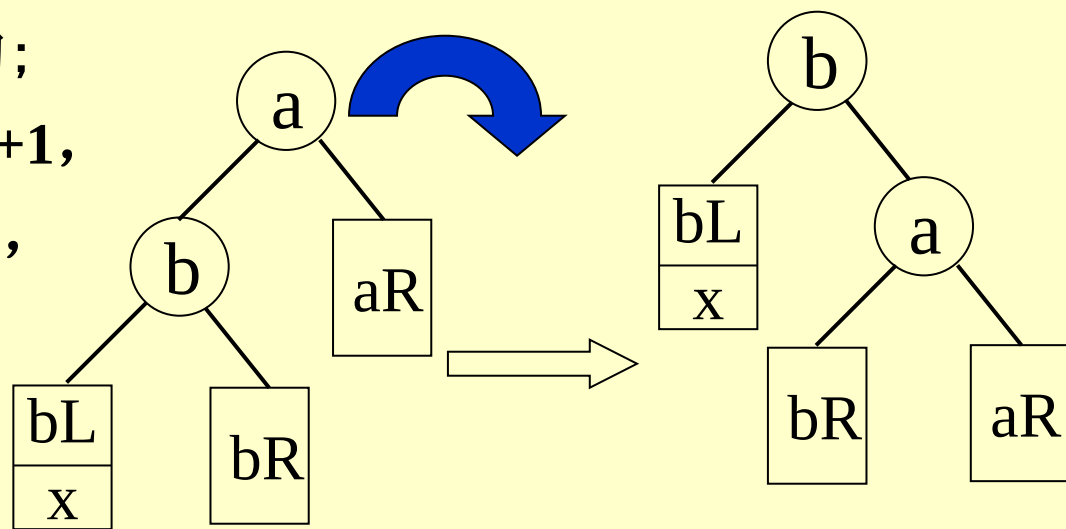
(3) 插入后各结点的平衡因子分析

① 旋转前的平衡因子

设插入后b的左子树的深度为 H_{bL} ，则其右子树的深度为 $H_{bL}-1$ ；a的左子树的深度为 $H_{bL}+1$ 。a的平衡因子为2，则a的右子树的深度为： $H_{aR}=H_{bL}+1-2=H_{bL}-1$ 。

② 旋转后的平衡因子

- a的右子树没有变，而左子树是b的右子树，则平衡因子是： $H_{aL}-H_{aR}=(H_{bL}-1)-(H_{bL}-1)=0$ ，即a是平衡的，以a为根的子树的深度是 H_{bL}
- b的左子树没有变化，右子树是以a为根的子树，则平衡因子是： $H_{bL}-H_{bR}=0$ ，即b也是平衡的；
- 以b为根的子树的深度是 $H_{bL}+1$ ，与插入前a的子树的深度相同，则该子树的上层各结点的平衡因子没有变化，则整棵树旋转后是平衡的。



LL型平衡化旋转

(4) 旋转算法

```
void R_Rotate(BSTree *p)
```

```
{//对以*p为根的二叉排序树做右旋处理，处  
理之后p指向新的树根结点， *p为返回值。
```

```
    BSTree lc;
```

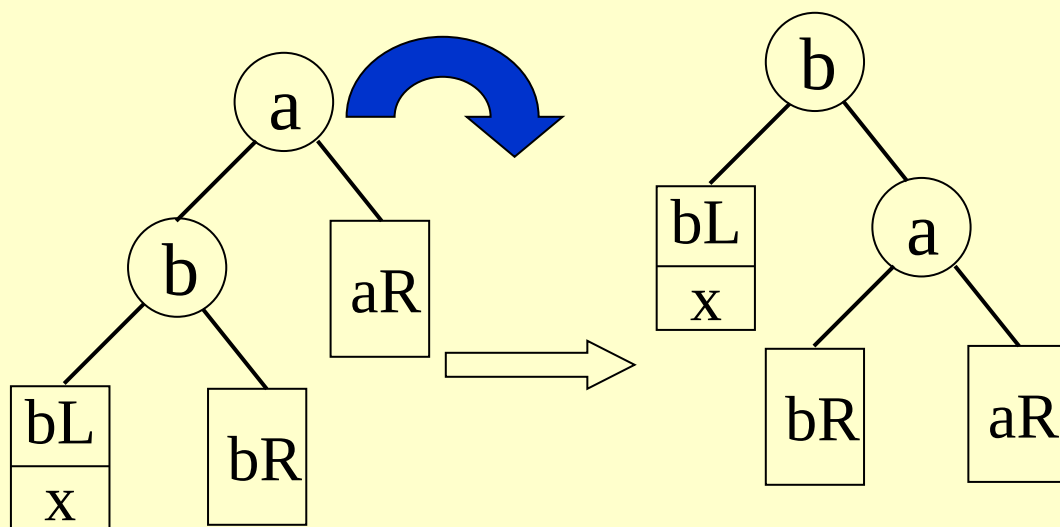
```
    lc = (*p)->lchild;
```

```
    _____;
```

```
    _____;
```

```
    _____;
```

```
}
```



注意次序以防断结点

LL型平衡化旋转

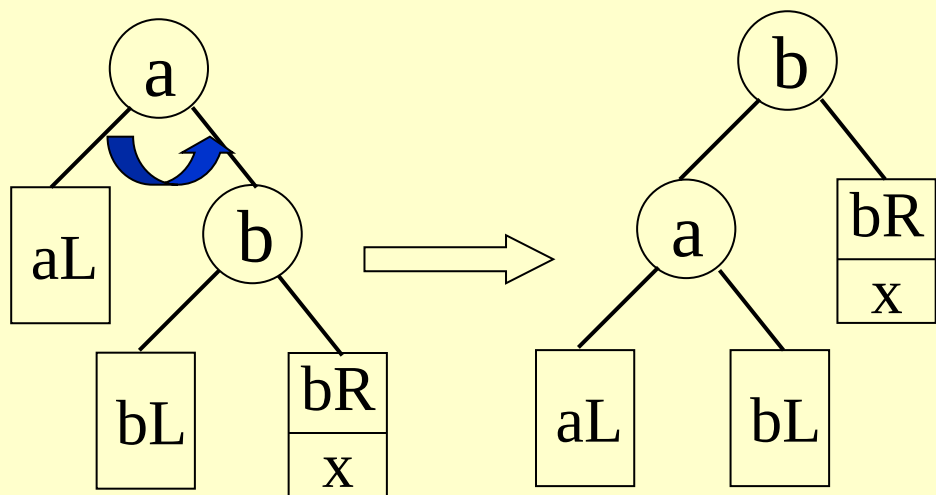
2 RR型平衡化旋转

(1) 失衡原因

在结点a的右孩子的右子树上进行插入，插入使结点a失去平衡。要进行一次逆时针旋转，和LL型平衡化旋转正好对称。

(2) 平衡化旋转方法 左旋

设b是a的右孩子，通过逆时针旋转实现。用b取代a的位置，a作为b的左子树的根结点，b原来的左子树作为a的右子树。



RR型平衡化旋转

(3) 旋转算法

```
void L_Rotate(BSTree *p)
```

{//对以*p为根的二叉排序树做左旋处理，处理之后p指向新的树根结点，*p为返回值。

```
    BSTree rc;
```

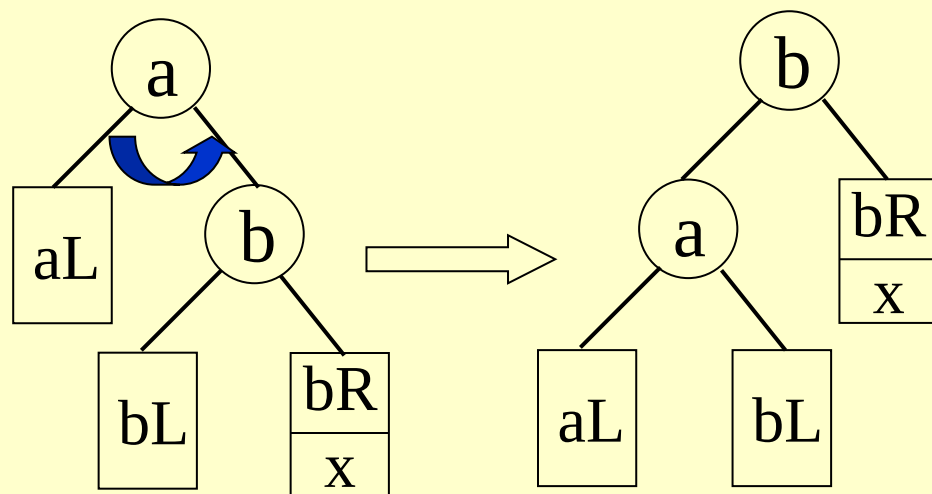
```
    rc = (*p)->rchild;
```

```
    _____;
```

```
    _____;
```

```
    _____;
```

```
}
```

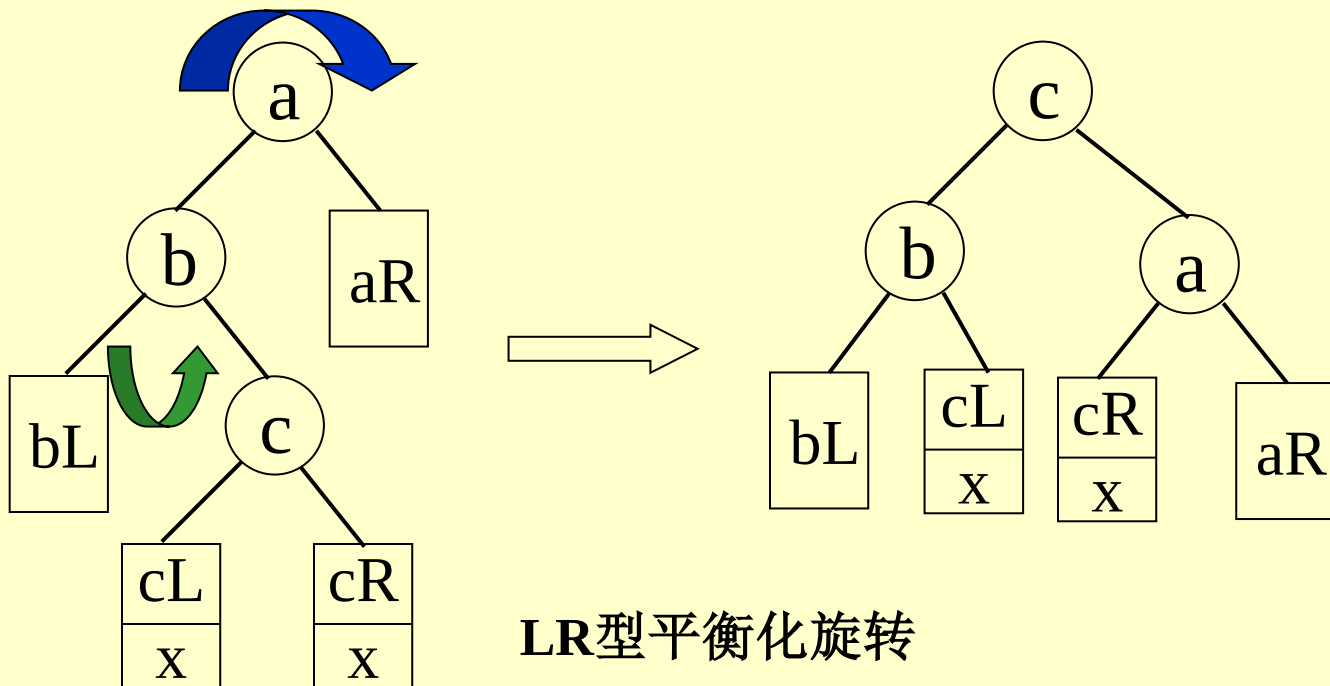


RR型平衡化旋转

3 LR型平衡化旋转

(1) 失衡原因

在结点a的左孩子的右子树上进行插入，插入使结点a失去平衡。a插入前的平衡因子是1，插入后a的平衡因子是2。设b是a的左孩子，c为b的右孩子，b在插入前的平衡因子只能是0，插入后的平衡因子是-1；c在插入前的平衡因子只能是0，否则，c就是失衡结点。

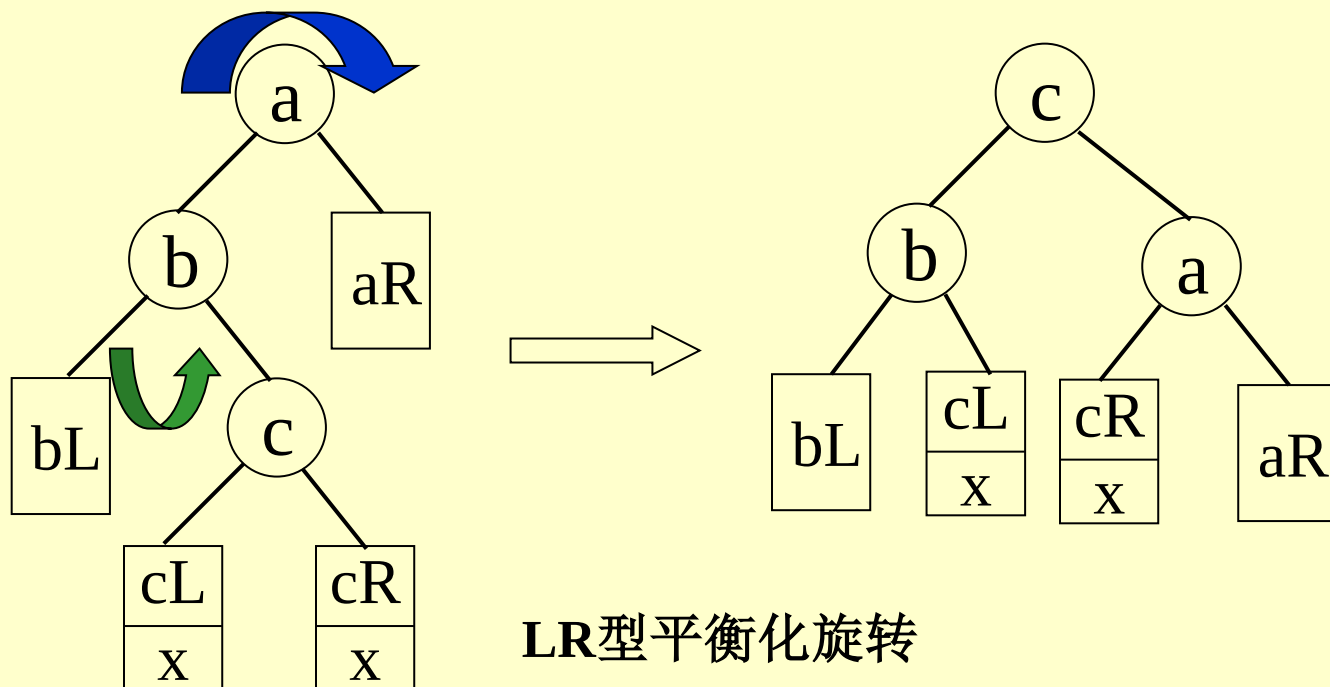


(2) 插入后结点c的平衡因子的变化分析

- ① 插入后c的平衡因子是1：即在c的左子树上插入。设c的左子树的深度为 H_{cL} ，则右子树的深度为 $H_{cL}-1$ ；b插入后的平衡因子是-1，则b的左子树的深度为 H_{cL} ，以b为根的子树的深度是 $H_{cL}+2$ 。因插入后a的平衡因子是2，则a的右子树的深度是 H_{cL} 。
- ② 插入后c的平衡因子是0：c本身是插入结点。设c的左子树的深度为 H_{cL} ，则右子树的深度也是 H_{cL} ；因b插入后的平衡因子是-1，则b的左子树的深度为 H_{cL} ，以b为根的子树的深度是 $H_{cL}+2$ ；插入后a的平衡因子是2，则a的右子树的深度是 H_{cL} 。
- ③ 插入后c的平衡因子是-1：即在c的右子树上插入。设c的左子树的深度为 H_{cL} ，则右子树的深度为 $H_{cL}+1$ ，以c为根的子树的深度是 $H_{cL}+2$ ；因b插入后的平衡因子是-1，则b的左子树的深度为 $H_{cL}+1$ ，以b为根的子树的深度是 $H_{cL}+3$ ；则a的右子树的深度是 $H_{cL}+1$ 。

(3) 平衡化旋转方法

先以**b**进行一次**逆时针旋转**(将以b为根的子树旋转为以c为根),
再以**a**进行一次**顺时针旋转**。将整棵子树**旋转**为以c为根, b是c的
左子树, a是c的右子树; c的右子树移到a的左子树位置, c的左
子树移到b的右子树位置。



LR型平衡化旋转

(4) 旋转后各结点(a,b,c)平衡因子分析

① 旋转前 (插入后)c的平衡因子是1:

a的左子树深度为 $H_{cL}-1$ ，其右子树没有变化，深度是 H_{cL} ，则a的平衡因子是-1；b的左子树没有变化，深度为 H_{cL} ，右子树是c旋转前的左子树，深度为 H_{cL} ，则b的平衡因子是0；c的左、右子树分别是以b和a为根的子树，则c的平衡因子是0。

② 旋转前 (插入后)c的平衡因子是0:

旋转后a, b, c的平衡因子都是0。

③ 旋转前 (插入后)c的平衡因子是-1:

旋转后a, b, c的平衡因子分别是0, -1, 0。

综上整棵树旋转后是平衡的。

```

void LeftBalance(BSTree * T)
{   BSTree lc,rd;
    lc = (*T)->lchild;
    switch (lc->bf)
    {   case LH:    //LL
        (*T)->bf = lc->bf = EH;
        _____;    break;
    case RH:    //LR
        rd = lc->rchild;
        switch(rd->bf) //各结点bf赋值再旋转
        {   case LH:
            (*T)->bf = RH;
            lc->bf = EH; break;
            case EH:
            (*T)->bf = lc->bf = EH; break;
            case RH:
            (*T)->bf = EH;
            lc->bf = LH; break;
        }
        rd->bf = EH;
        L_Rotate(_____); //左旋
        _____; break; //右旋
    }
}

```

(5) 旋转算法 先左旋后右旋

综合考虑LL与LR两种情况定义函数

```
void LeftBalance(BSTree * T)
```

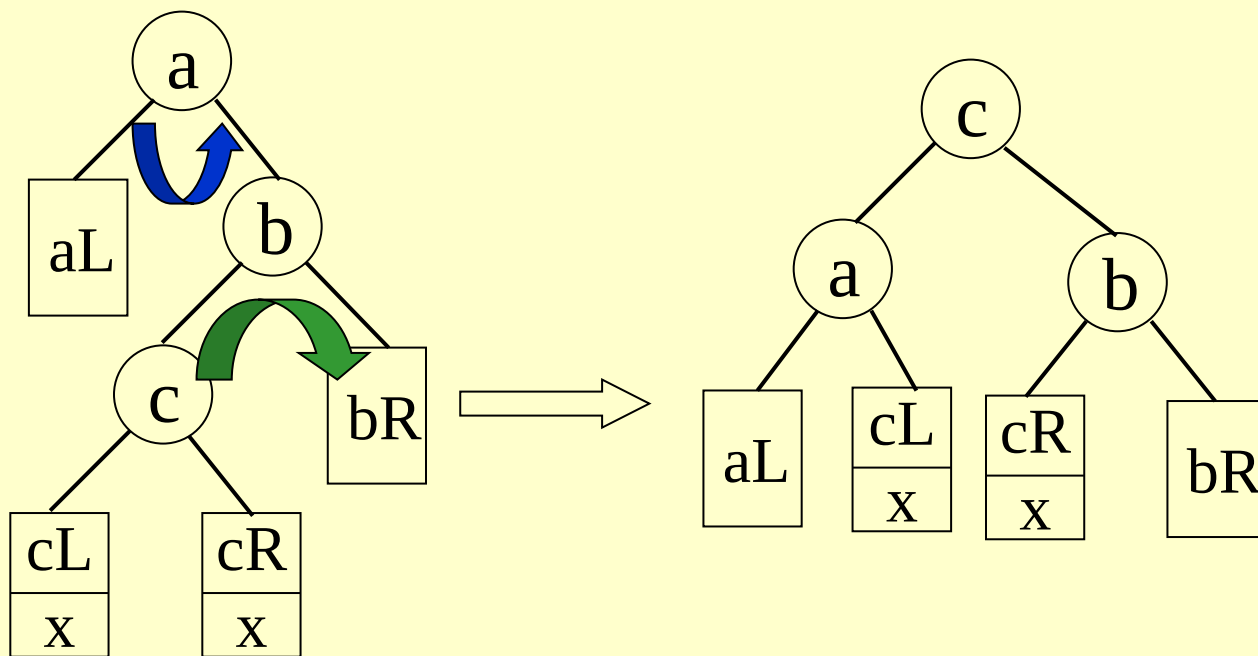
//对以指针 T 所指向结点为根结点的
二叉树作左子树的平衡处理，
令指针 T 指向新的根结点

//查看以 T 的左子树为根结点的子树，
失去平衡的原因，如果 bf 值为 1，
则说明添加在左子树为根结点的左子
树中，需要对其进行右旋处理；反之，
如果 bf 值为 -1，说明添加在以左子
树为根结点的右子树中，需要进行双
向先左旋后右旋的处理

4 RL型平衡化旋转

(1) 失衡原因

在结点a的右孩子的左子树上进行插入，插入使结点a失去平衡，与LR型正好对称。对于结点a，插入前的平衡因子是-1，插入后a的平衡因子是-2。设b是a的右孩子，c为b的左孩子，b在插入前的平衡因子只能是0，插入后的平衡因子是1；同样，c在插入前的平衡因子只能是0，否则，c就是失衡结点。



RL型平衡化旋转

(2) 插入后结点c的平衡因子的变化分析

① 插入后c的平衡因子是1：在c的左子树上插入。设c的左子树的深度为 H_{cL} ，则右子树的深度为 $H_{cL}-1$ 。

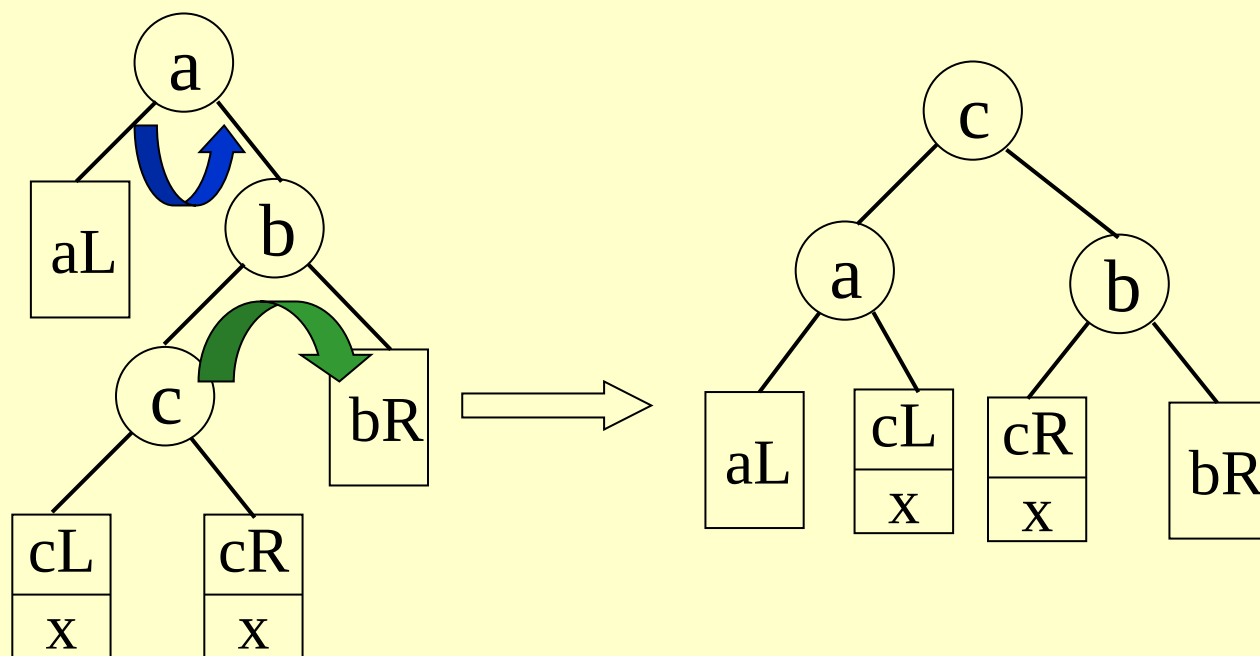
因b插入后的平衡因子是1，则其右子树的深度为 H_{cL} ，以b为根的子树的深度是 $H_{cL}+2$ ；因插入后a的平衡因子是-2，则a的左子树的深度是 H_{cL} 。

② 插入后c的平衡因子是0：c本身是插入结点。设c的左子树的深度为 H_{cL} ，则右子树的深度也是 H_{cL} ；因b插入后的平衡因子是1，则b的右子树的深度为 H_{cL} ，以b为根的子树的深度是 $H_{cL}+2$ ；因插入后a的平衡因子是-2，则a的左子树的深度是 H_{cL} 。

③ 插入后c的平衡因子是-1：在c的右子树上插入。设c的左子树的深度为 H_{cL} ，则右子树的深度为 $H_{cL}+1$ ，以c为根的子树的深度是 $H_{cL}+2$ ；因b插入后的平衡因子是1，则b的右子树的深度为 $H_{cL}+1$ ，以b为根的子树的深度是 $H_{cL}+3$ ；则a的右子树的深度是 $H_{cL}+1$ 。

(3) 平衡化旋转方法

先以**b**进行一次顺时针旋转，再以**a**进行一次逆时针旋转。
即将整棵子树(以**a**为根)旋转为以**c**为根，**a**是**c**的左子树，**b**是**c**的右子树；**c**的右子树移到**b**的左子树位置，**c**的左子树移到**a**的右子树位置。



RL型平衡化旋转

(4) 旋转后各结点(a,b,c)的平衡因子分析

① 旋转前 (插入后)c的平衡因子是1:

a的左子树没有变化, 深度是 H_{cL} , 右子树是c旋转前的左子树, 深度为 H_{cL} , 则a的平衡因子是0; b的右子树没有变化, 深度为 H_{cL} , 左子树是c旋转前的右子树, 深度为 $H_{cL}-1$, 则b的平衡因子是-1; c的左、右子树分别是以a 和b为根的子树, 则c的平衡因子是0。

② 旋转前 (插入后)c的平衡因子是0:

旋转后a, b, c的平衡因子都是0。

③ 旋转前 (插入后)c的平衡因子是-1:

旋转后a, b, c的平衡因子分别是1, 0, 0。

综上所述, 即整棵树旋转后是平衡的。

```

void RightBalance(BSTree* T)
{ BSTree lc,rd;
  lc= (*T)->rchild;
  switch (lc->bf)
  { case RH:
      (*T)->bf = lc->bf = EH;
      _____ ;      break;
    case LH:
      rd = lc->lchild;
      switch(rd->bf)
      {
        case LH:
          (*T)->bf = EH;
          lc->bf = RH;      break;
        case EH:
          (*T)->bf = lc->bf = EH;      break;
        case RH:
          (*T)->bf = EH;
          lc->bf = LH;      break;
      }
      rd->bf = EH;
      R_Rotate(_____);
      _____ ;      break;
  }
}
}

```

(5) 旋转算法 先右旋后左旋

综合考虑RR与RL两种情况定义函数

```
void RightBalance(BSTree* T)
```

//对以指针 T 所指向结点为根结点的二叉树作右子树的平衡处理，令指针 T 指向新的根结点

//查看以 T 的左子树为根结点的子树，失去平衡的原因，如果 bf 值为 1，则说明添加在左子树为根结点的左子树中，需要对其进行右旋处理；反之，如果 bf 值为 -1，说明添加在以左子树为根结点的右子树中，需要进行双向先左旋后右旋的处理

平衡二叉排序树的插入

平衡二叉排序树的插入操作实际上是在二叉排序插入的基础上完成以下工作：

- (1): 判别插入结点后的二叉排序树是否产生不平衡？
- (2): 找出失去平衡的最小子树；
- (3): 判断旋转类型，然后做相应调整。

失衡的最小子树的根结点a在插入前的平衡因子不为0，且是离插入结点最近的平衡因子不为0的结点的。

若a失衡，从a到插入点的路径上的所有结点的平衡因子都会发生变化，在该路径上还有一个结点的平衡因子不为0且该结点插入后没有失衡，其平衡因子只能是由1到0或由-1到0，以该结点为根的子树深度不变。该结点的所有祖先结点的平衡因子也不变，更不会失衡。

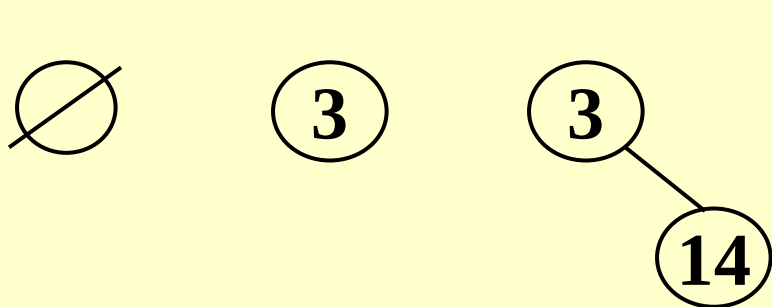
1 算法思想(插入结点的步骤)

- ① 按照二叉排序树的定义，将结点s插入；
- ② 在查找结点s的插入位置的过程中，记录离结点s最近且平衡因子不为0的结点a，若该结点不存在，则结点a指向根结点；
- ③ 修改结点a到结点s路径上所有结点的；
- ④ 判断是否产生不平衡，若不平衡，则确定旋转类型并做相应调整。

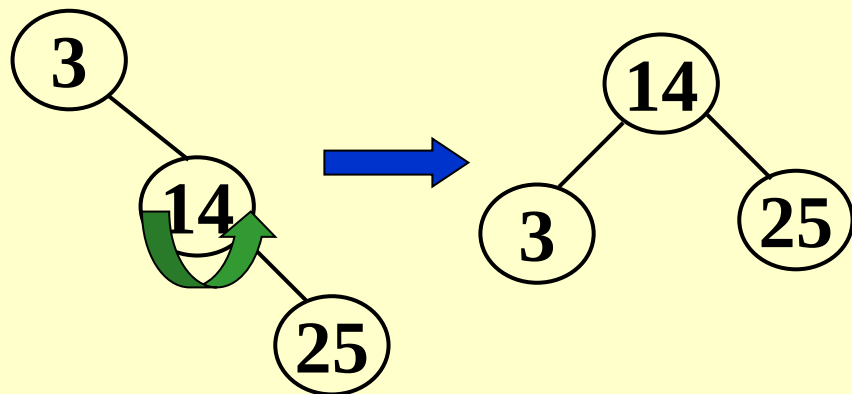
2 算法实现

```
int InsertAVL(BSTree* T,ElemType e,bool* taller)
```

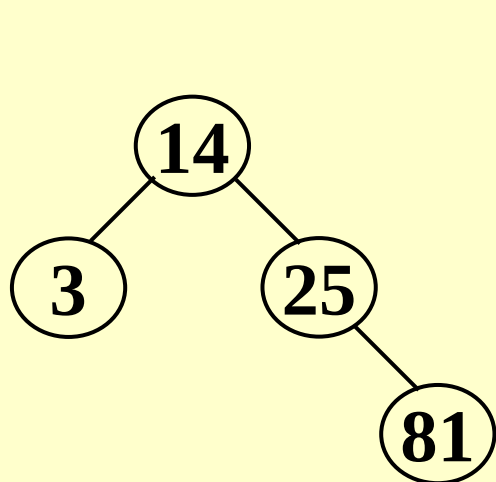
例： 设要构造的平衡二叉树中各结点的值分别是(3, 14, 25, 81, 44)，平衡二叉树的构造过程如图9-11所示。



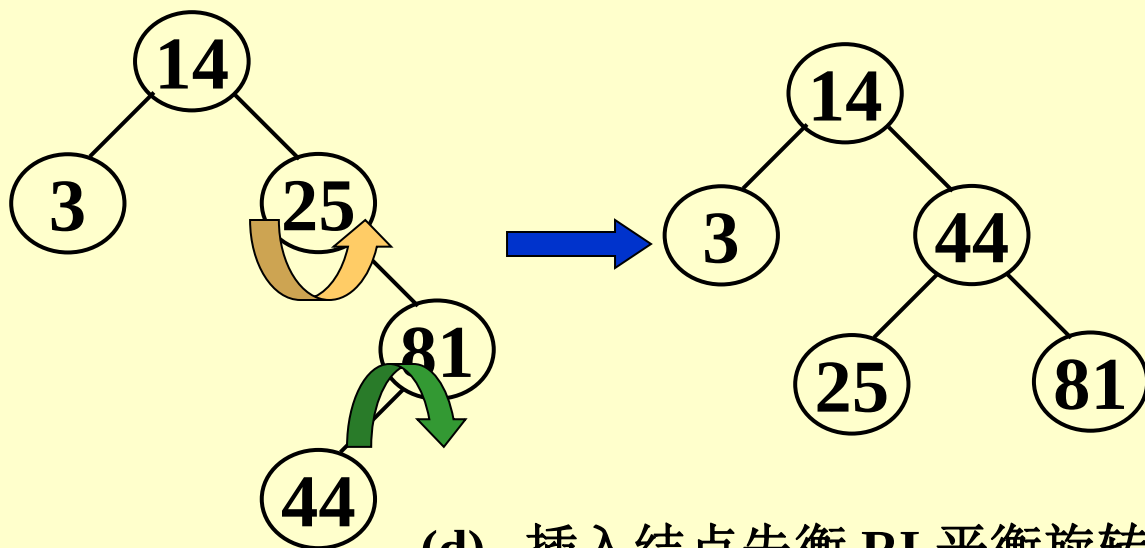
(a) 插入不超过两个结点



(b) 插入新结点失衡,RR平衡旋转



(c) 插入新结点未失衡



(d) 插入结点失衡,RL平衡旋转

平衡二叉树的构造过程

练习： 设要构造的平衡二叉树中各结点的值分别是
(15,3,7,10,9,8), 平衡二叉树的构造过程，给出旋转类型。

Experiment 6.6 AVL Tree Implementation

Problems:

//对以 p 为根结点的二叉树做右旋处理，令 p 指针指向新的树根结点

void R_Rotate(BSTree* p);

//对以 p 为根结点的二叉树做左旋处理，令 p 指针指向新的树根结点

void L_Rotate(BSTree* p);

//对以指针 T 所指向结点为根结点的二叉树作左子树的平衡处理，
令指针 T 指向新的根结点

void LeftBalance(BSTree * T);

//右子树的平衡处理同左子树的平衡处理完全类似

void RightBalance(BSTree* T);

int InsertAVL(BSTree* T,ElemType e,bool* taller);

//判断现有平衡二叉树中是否已经具有数据域为 e 的结点

bool FindNode(BSTree root,ElemType e,BSTree* pos);

//中序遍历平衡二叉树

void InorderTra(BSTree root);

110.平衡二叉树

给定一个二叉树，判断它是否是高度平衡的二叉树。

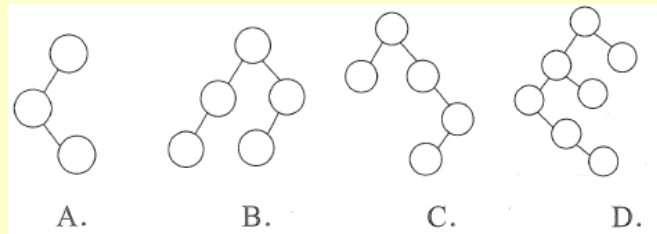
1、含有20个结点的平衡二叉树的最大深度为（ ）。

- A、 4 B、 5 C、 6 D、 7

11、具有5层结点的AVL至少有（ ）结点。

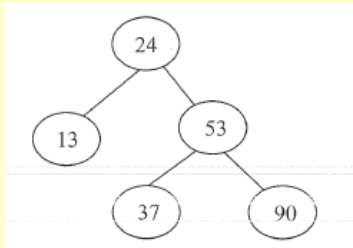
- A、 10 B、 12
C、 15 D、 17

2、【2009统考真题】下列二叉排序树中，满足平衡二叉树定义的是（ ）。



3、【2010统考真题】在如下图所示的平衡二叉树中插入关键字48后得到一棵新平衡二叉树，在新平衡二叉树中，关键字37所在结点的左、右子结点中保存的关键字分别是（ ）。

- A、 13, 48 B、 24, 48
C、 24, 53 D、 24, 90



4、【2012统考真题】若平衡二叉树的高度为6，且所有非叶子结点的平衡因子均为1，则该平衡二叉树的结点总数为（ ）。

- A、 12 B、 20 C、 32 D、 33

5、【2013统考真题】若将关键字1， 2， 3， 4， 5， 6， 7依次插入初始为空的平衡二叉树T，则T中平衡因子为0的分支结点的个数是（ ）。

- A、 0 B、 1 C、 2 D、 3